ML_Task_3(Kmeans)

June 11, 2026

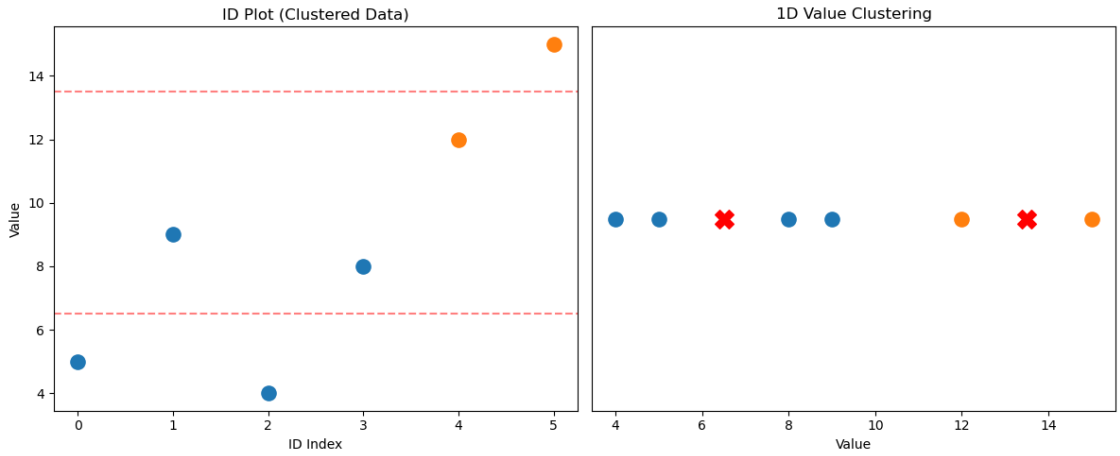
0.1 1) Apply k means clustering on ID Dataset [5, 9, 4, 8, 12, 15] and make the plot

[12]:
```
#Importing Libraries
import os
os.environ["OMP_NUM_THREADS"] = "1"
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Data
D = np.array([5, 9, 4, 8, 12, 15]).reshape(-1, 1)

# KMeans
kmeans = KMeans(n_clusters=2, random_state=0, n_init=10)
labels = kmeans.fit_predict(D)
centroids = kmeans.cluster_centers_.flatten()

print("Data & Cluster:")
for d, l in zip(D, labels):
    print(d[0], "→ Cluster", l+1)

print("\nCentroids:", centroids)

ids = np.arange(len(D))

# -------------------------
# Plot 1: ID Plot (FIXED)
# -------------------------
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

for i in range(len(D)):
    plt.scatter(ids[i], D[i],
                c=f"C{labels[i]}",
                s=120)
```

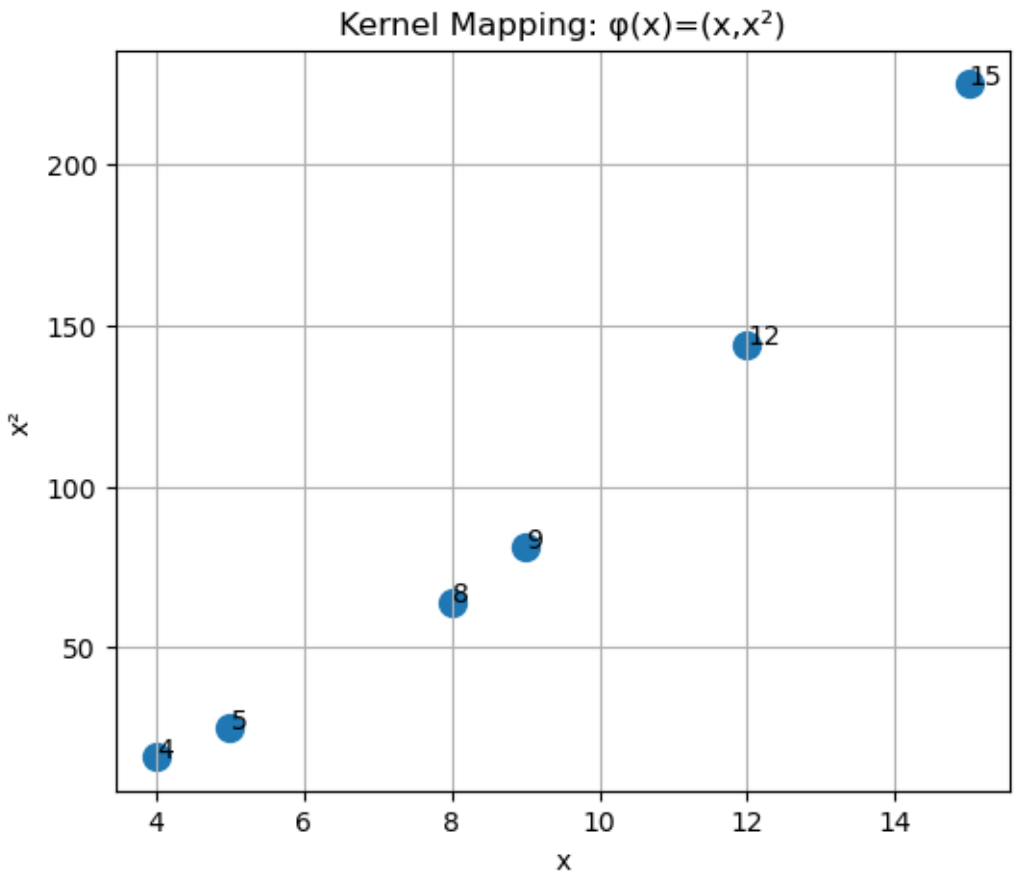
```
# FIX: correct centroid representation in ID space
for c in centroids:
    plt.axhline(y=c, color='red', linestyle='--', alpha=0.5)

plt.title("ID Plot (Clustered Data)")
plt.xlabel("ID Index")
plt.ylabel("Value")

# -------------------------
# Plot 2: 1D Value Space
# -------------------------
plt.subplot(1,2,2)

for i in range(len(D)):
    plt.scatter(D[i], 0,
                c=f"C{labels[i]}",
                s=120)

plt.scatter(centroids, np.zeros_like(centroids),
            c="red", marker="X", s=200)

plt.title("1D Value Clustering")
plt.xlabel("Value")
plt.yticks([])

plt.tight_layout()
plt.show()
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

```
Data & Cluster:
5 → Cluster 1
9 → Cluster 1
4 → Cluster 1
8 → Cluster 1
12 → Cluster 2
15 → Cluster 2

Centroids: [ 6.5 13.5]
```



[1]:
```
import numpy as np
import matplotlib.pyplot as plt

D = np.array([5, 9, 4, 8, 12, 15])

# Kernel mapping
X = D
Y = D**2

plt.figure(figsize=(6,5))

plt.scatter(X, Y, s=100)

for i in range(len(D)):
    plt.annotate(str(D[i]), (X[i], Y[i]))

plt.xlabel("x")
plt.ylabel("x²")
plt.title("Kernel Mapping:  (x)=(x,x²)")
plt.grid(True)

plt.show()
```



0.2 2) Apply k means clustering on 2D Dataset (5,4) (8,8),(9,12),(10,15),(16,10) and make the plot.

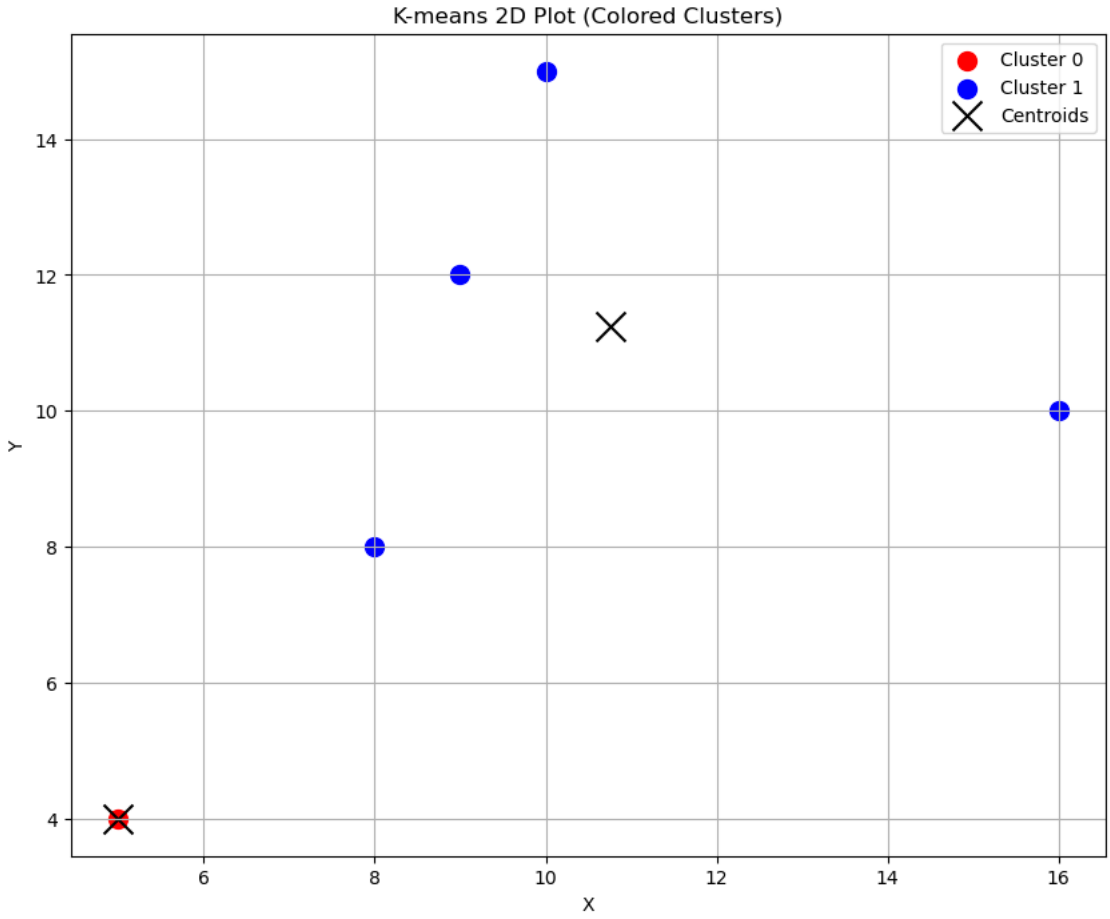
[27]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# -------------------------
# DATA
# -------------------------
X = np.array([
    [5, 4],
    [8, 8],
    [9, 12],
    [10, 15],
    [16, 10]
])

# -------------------------
```

```python
# PARAMETERS
# -------------------------
K = 2
max_iters = 100

centroids = np.array([
    [5, 4],
    [8, 8]
])

# -------------------------
# DISTANCE
# -------------------------
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# -------------------------
# K-MEANS LOOP
# -------------------------
for iteration in range(max_iters):

    clusters = [[] for _ in range(K)]
    labels = []

    # Assign clusters
    for point in X:
        distances = [euclidean_distance(point, c) for c in centroids]
        idx = np.argmin(distances)
        clusters[idx].append(point)
        labels.append(idx)

    labels = np.array(labels)

    # Update centroids
    new_centroids = []
    for cluster in clusters:
        if len(cluster) > 0:
            new_centroids.append(np.mean(cluster, axis=0))
        else:
            new_centroids.append(np.zeros(2))

    new_centroids = np.array(new_centroids)

    # Convergence check
    if np.allclose(centroids, new_centroids):
        print(f"Converged at iteration {iteration}")
        break
```

```
    centroids = new_centroids

# -------------------------
# RESULTS
# -------------------------
print("\nFinal Centroids:")
print(centroids)

print("\nFinal Labels:")
print(labels)

# -------------------------
# TABLE OUTPUT (CLUSTERS)
# -------------------------

data = pd.DataFrame(X, columns=["X", "Y"])
data["Cluster"] = labels

# Sort by cluster for readability
data = data.sort_values("Cluster").reset_index(drop=True)

print("\nCluster Assignment Table:")
print(data)

# -------------------------
# PLOT
# -------------------------
plt.figure(figsize=(10,8))

colors = ['red', 'blue', 'green', 'purple', 'orange']

for i in range(K):
    cluster = X[labels == i]
    plt.scatter(
        cluster[:, 0],
        cluster[:, 1],
        color=colors[i],
        label=f"Cluster {i}",
        s=100
    )

# Centroids (black X for clarity)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='x',
```

```
    s=250,
    color='black',
    label='Centroids'
)

plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.title("K-means 2D Plot (Colored Clusters)")
plt.grid(True)
plt.show()
```

```
Converged at iteration 1

Final Centroids:
[[ 5.    4.  ]
 [10.75 11.25]]

Final Labels:
[0 1 1 1 1]

Cluster Assignment Table:
    X   Y  Cluster
0   5   4        0
1   8   8        1
2   9  12        1
3  10  15        1
4  16  10        1
```


K-means 2D Plot (Colored Clusters)
Cluster 0
Cluster 1
Centroids
14
12
10
8
6
4
Y
6
8
10
12
14
16
X